



Escola Regional de Computação
Bahia, Alagoas e Sergipe

ProgBASE

Maratona de Programação



Realização: Organização:



Sistema FIEB



Apoio:



A) Sobre a entrada:

- 1) A entrada de seu programa deve ser lida da entrada padrão.
- 2) A entrada é composta por vários casos de teste, cada um descrito em um número de linhas que depende do problema.
- 3) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- 4) Cada linha, incluindo a última, contém o caractere final-de-linha.
- 5) O final da entrada coincide com o final do arquivo.

B) Sobre o seu código:

- 1) O nome do arquivo de seu programa é o nome do problema seguido da extensão da linguagem escolhida.
- 2) O seu programa deve ser escrito em uma das linguagens permitidas: Java, C, C++ ou Python 2 ou 3.
- 3) Um problema pode ser resolvido em uma linguagem diferente de outro, se desejar.
- 4) O seu programa deve ler a entrada do console (entrada padrão).
- 5) O seu programa deve gerar a saída em console (saída padrão).

C) Sobre a saída:

- 1) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- 2) Cada linha, incluindo a última, deve conter o caractere final-de-linha.

D) Sobre a competição:

- 1) Linguagens aceitas:
 - a) C, extensão “.c” → `gcc -std=c99 -O2 -lm`
 - b) C++, extensão “.cpp” → `g++ -std=c++20 -O2 -lm`
 - c) Java, extensão “.java” → `LC_ALL=en_US.UTF-8 java .. +2s` para limite de tempo
 - e) Python 3.9, extensão “.py” → `python3 .. +1s` para limite de tempo
- 2) As questões, embora escritas de forma original pela equipe organizadora, podem ser baseadas em problemas de mesma categoria já existentes.

Problema A
A1, A2 ou A3?
Arquivo: a.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

A Carteira Nacional de Habilitação (CNH) traz a categoria à qual o motorista é habilitado, na forma de letras: A, B, C, D e E. Para qualquer habilitação há a necessidade de exames médicos, teóricos e práticos, específicos de cada categoria. A tabela abaixo indica cada categoria, limites e obrigações apresentando as diferenças.

Categoria	Idade mínima	limite peso	limite lugares	Necessidade
A	18 anos	Veículos 2 e 3 rodas		
B	18 anos	3.500 kg	8 lugares	
C	21 anos	3.500 kg	8 lugares	1 ano cat B
D	21 anos	3.500 kg	> 8 lugares	2 anos cat B
E	21 anos	> 3.500 kg	> 8 lugares	1 ano cat C

Mas estas categorias podem ser alteradas. Por exemplo em 2017 surgiu a categoria D1, ela é uma subcategoria da D entre a C e a D. Destinada para condução de veículos escolares com mais de 8 lugares e até 16 lugares, mas não permite a condução de ônibus. Neste contexto há a ideia de trazer de volta as divisões da categoria A. No passado eram A1, A2 e A3 que representava o limite em cilindrada para as motos. Muitos defendem que um piloto habilitado com uma moto de 125 cilindradas não está habilitado automaticamente para pilotar uma de 2000 cilindradas. O PL 3245/15 do deputado Ronaldo Fonseca propõe a alteração do código de trânsito para uma nova divisão: A1 para motos de até 125cc, A2 para motos de até 300cc e A3 para motos de cilindradas livres. A você foi pedido para indicar a categoria mínima da habilitação para pilotar uma dada as cilindradas fornecidas.

Entrada:

Cada caso de entrada possui um valor V de cilindradas de moto ($50 < V < 3000$) acompanhado da unidade: "cc". Não vamos considerar aqui motos especiais, por exemplo as movidas a turbinas de jato.

Saída:

Em uma linha apenas a categoria adequada para a habilitação de acordo com as regras propostas pelo PL.

Exemplo de entrada:

Saída do Exemplo de entrada:

50cc	A1
180cc	A2
125cc	A1
750cc	A3

Curiosidade: Em 2022 foram definidas novas categorias para a habilitação: ACC, A, A1, B, B1, C, C1, D, D1, BE, CE, C1E, DE e D1E.

Problema B

Bolas Fora

Arquivo: *bolas.[c|cpp|java|py]*

Limite de tempo: 1 s

O Problema:

Em Nlogônia existe o jogo Bolas Fora, que é um jogo realizado em n rodadas para 2 jogadores. São distribuídas em um plano $4n$ bolas de gude. A cada rodada um jogador retira 2 bolas e é adicionado ao seu placar a distância entre as bolas retiradas. Ganha o jogador que ficar com o menor placar. Seu colega sabe que se ele for o primeiro a começar a jogar ele consegue ganhar (ou ao menos empatar) mesmo que ambos joguem de forma ótima. Dada a distribuição de bolas ele quer que você indique qual o placar máximo que ele pode conseguir se sempre jogar de forma ótima (pegar as duas bolas de gude de menor distância do conjunto que está disponível na mesa).

Entrada:

A entrada consiste de um valor inteiro n ($1 \leq n \leq 200$) o número de rodadas. Seguida de $4n$ linhas onde cada linha contém duas coordenadas, em ponto flutuante, (x, y) do plano, para cada bola. ($-1 \leq x, y \leq 1$)

Saída

A saída representa o valor máximo que o primeiro jogador terá no placar, com 2 casas decimais, caso jogue de forma ótima. (Considere que o adversário também jogue de forma ótima).

Exemplo de entrada:

```
2
-0.1 0.1
0.5 0.5
0 0
0.2 -0.3
-0.3 -0.4
0.5 -0.5
0.1 1
-1 -0.5
```

Saída do Exemplo de entrada:

0.78

```
1
1 1
-1 1
-1 -1
1 -1
```

2.00

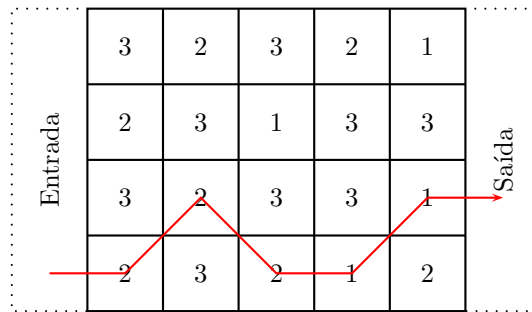
Problema C

Ciclismo de Obstáculo

Arquivo: *ciclismo*.*[c|cpp|java|py]*
 Limite de tempo: 1 s

O Problema:

Em uma região montanhosa e árida entre dois rios (razoavelmente paralelos) foi criada uma corrida de obstáculos para bicicleta. Sendo uma região árida, então qualquer desnível seja subida seja descida é um obstáculo difícil por haver pedras soltas, ou seja, se num trecho tiver de subir 3m e depois descer 3m, será bem pior que outro onde tenha de subir 2m e depois subir 2m, o desnível total de 4m é menor que o desnível total de 6m do primeiro trecho. O mapa da região foi dividido em áreas por suas alturas. A figura abaixo demonstra um relevo indicando a altura em cada área em uma divisão de 4 corredores por 5 intervalos, considerando que a entrada e a saída estão em altura 0. Observe que de uma área é possível seguir à frente na próxima área do mesmo corredor ou mudar para uma das duas áreas vizinhas da área à frente (quando houver).



No quadro a seta indica um possível caminho onde a soma total dos desníveis é de apenas 4 (será que há mais?), não há caminho cuja soma de desnível seja menor. Seu colega pediu que você analise o terreno e diga qual será o menor esforço possível (ou seja a menor soma dos desniveis) para qualquer caminho entre a entrada e a saída de um determinado relevo.

Entrada:

O caso de entrada representa um terreno. A primeira linha são dois inteiros, o primeiro (c , $0 < c < 1000$) informa o número de corredores o segundo, (a , $0 < a < 1000$) o número de áreas em que são divididos os corredores. Segue então a linhas com c valores inteiros cada linha ($0 < c_i < 10$) em cada linha, valores descritos em metros).

Saída:

Para o caso de entrada a soma total, em seu mínimo possível, dos desníveis ao percorrer áreas da entrada à saída.

Exemplo de entrada:

```
4 5
2 3 2 3
3 2 3 2
2 3 1 3
1 3 3 2
2 1 3 1
```

Saída do Exemplo de entrada:

4

```
2 3
1 2
2 1
1 2
```

2

Problema D

Decodificando Morse

Arquivo: *decodificando.[c|cpp|java|py]*

Limite de tempo: 1 s

O Problema:

Uma das mais antigas formas de comunicação à distância é o telégrafo. Antes do telefone, antes de tudo, já tinha o telégrafo. O telégrafo se baseava em um sinal. Quando a origem gerava o sinal em todos os receptores acionava uma chave. Então a codificação era baseada em sinal longo e sinal curto. Observe que isto também era usada na forma de luz (abrindo e fechando a fonte de luz na origem, longo e curto) e também via rádio (existência de portadora, longo e curto). Hoje em dia se mantém este tipo de comunicação como hobby entre os adeptos do radio-amadorismo. Na codificação morse, um sinal curto é definido como “.” e um sinal longo como “-”. Cada letra e número tem uma codificação específica. Por exemplo: “a = .-”, “1 = .-...” e “o = ---”. Então quando recebe-se “.- .-... ---” decodifica-se como “alo”. Para separar palavras usamos “/”: “.- .-... --- / .-... --- .-... .-” representa “alo lola”. Um colega lhe pediu que decodifique os textos que ele recebe em código morse.

Segue a tabela do código:

A	.-	N	-.	1	-----
B	-...	O	---	2	..---
C	-.-.	P	.-..	3	...--
D	-..	Q	--.-	4	-
E	.	R	-.-	5	
F	..-.	S	...	6	-....
G	--.	T	-	7	--...
H		U	..-	8	---..
I	..	V	...-	9	----.
J	.----	W	.-.	0	-----
K	-.-	X	-..-		
L	.-...	Y	-.--		
M	--	Z	--..		

Entrada:

A entrada possui vários casos de teste. Cada caso de teste é uma linha com codificação morse de no máximo 500 caracteres, cada letra numa palavra é separada por um espaço em branco ' ' em sua codificação morse, e cada palavra é separada de outra por “/” (uma barra diagonal dentro de dois espaços em brancos). Os casos de teste terminam com o final da entrada.

Saída:

Para cada caso de teste da entrada deve ser gerado na saída em uma linha o texto decodificado em ASCII minúsculas.

Exemplo de entrada:

```
.- .-... --- / .-... --- .-... .-  
.-.. -.- --- --. .... .- ... . / ..--- ----- ..--- ....-
```

Saída do exemplo de entrada:

```
alo lola  
progbase 2024
```

Problema E

Economia

Arquivo: *economia*.*[c|cpp|java|py]*

Limite de tempo: 1 s

O Problema:

A indústria sempre busca uma forma de economizar e aumentar as chances de lucro. Um exemplo é diminuir a quantidade de produtos empacotados e manter o preço. Desta forma um mesmo produto de indústrias distintas podem ser oferecido na mais variada forma de empacotamento e preços diversos, ficando difícil saber qual o mais barato. Um exemplo disto são os papéis higiênicos. Encontramos pacotes de 4, 8, 12 até 16 rolos, bem como tamanhos distintos de rolos, de 20m, 30m, 50m e até de 100m. Com isto seu amigo fica muito confuso na hora de comprar. Ele gosta de comprar o equivalente a 60 rolos de 30m, e quer o menor preço. Portanto seu amigo pediu para listar qual marca ou marcas oferece o menor preço para a compra.

Entrada:

A entrada contém vários casos de teste, cada caso de teste começa com um valor inteiro positivo N ($0 < N < 100$) indicando a quantidade de marcas de papel. As próximas N linhas contém: s o nome da marca (uma única palavra no padrão ascii em minúsculas de no máximo 10 letras), t o tamanho de um rolo do pacote em metros ($0 < t \leq 100$), p a quantidade de rolos do pacote ($0 < p \leq 20$) e r o valor do pacote como um número real em ponto fixo com 2 casas decimais ($0 < r \leq 64$). Os casos de testes terminam com $N = 0$ que não deve ser tratado. É garantido que sempre um número inteiro de pacotes irão inteirar o equivalente a 60 rolos de 30m.

Saída:

Para cada caso de teste deve ser impresso a(s) marca(s) que oferece(m) o menor preço. Quando mais de uma, uma em cada linha em ordem alfabética. Após cada caso de teste deve ser impressa uma linha em branco, inclusive após o último.

Exemplo de entrada:

```
3
nove 30 6 6.21
fluff 50 12 20.64
tutty 30 15 16.46
3
lua 50 6 10.29
fofo 20 5 3.43
tutty 30 4 4.20
0
```

Saída do exemplo de entrada:

```
fluff

fofo
lua
```

Problema F
Fair Play
Arquivo: *fair*.*[c|cpp|java|py]*
Limite de tempo: 1 s

O Problema:

Nos jogos “*Fair Play*” significa “Jogos Justos”, ou seja ambos os times devem jogar de forma limpa. Em torneios entre amigos, este termo assume outro aspecto, na divisão dos times a formação dos times deve ter a mesma força, evitando assim um time muito mais forte que o outro. Para isso, cada jogador recebe um *Índice de Força*. No final, o total dos índices para os times deve ter uma diferença mínima entre ambos. Por exemplo, sejam 10 jogadores com a seguinte distribuição de índices de força:

2 5 8 3 4 9 2 1 8 4

Podem ser divididos como:

2 5 8 4 4
9 2 1 8 3

Cada time ficou com força 23, iguais. Um jogo justo. Assim a você foi pedido, dada a distribuição das forças em um determinado conjunto de jogadores, dividir de forma que a diferença de força entre os dois times seja a mínima. Uma curiosidade, uma vez as forças equilibradas os times não precisam ter o mesmo número de jogadores. Seria o equivalente a um jogo de 8 jogadores de habilidade 1, contra 1 jogador de habilidade 8.

Entrada:

A entrada possui vários casos de teste. Cada caso de teste é representado em uma linha, iniciando com um valor inteiro n representando o números de jogadores ($2 \leq n < 100$) seguido por n valores inteiros v_i separados por um espaço em branco onde cada valor v_i ($0 < v_i < 100$) representa o índice de força do jogador i . Os casos de teste terminam com o fim da entrada.

Saida:

Para cada caso de teste deve ser impresso o valor da diferença mínima de força entre os times que podem ser formados representado por um inteiro apenas em uma linha cada.

Exemplo de entrada:

10 2 5 8 3 4 9 2 1 8 4
5 1 2 3 4 5
8 2 3 4 5 6 7 8

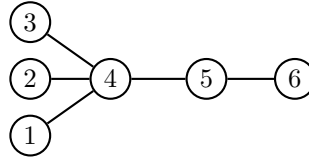
Saída do exemplo de entrada:

0
1
1

Problema G
Gato na Árvore
Arquivo: *gato.[c|cpp|java|py]*
Limite de tempo: 1 s

O Problema:

O “gato” no jargão coloquial representa uma ligação clandestina, seja em instalação elétrica, seja em ligação hidráulica, ou em outros contextos. Talvez próximo do jargão “bug” na língua inglesa. A árvore é um grafo conexo acíclico. Normalmente apresentamos uma árvore através das várias ligações entre seus nós. Por exemplo uma árvore de 6 nós pode ser representada por 5 ligações (aliás uma árvore de n nós tem $n - 1$ ligações): $\{1, 4\}$, $\{2, 4\}$, $\{3, 4\}$, $\{5, 4\}$, $\{5, 6\}$. Que pode ser representada abaixo na figura:



O “gato” neste caso acontece quando as ligações não formarem uma árvore. A você foi pedido que verifique as ligações propostas para uma árvore e indique se há ou não gato nas ligações.

Entrada:

A entrada possui vários casos de teste. Cada caso de teste é representado em uma linha, iniciando com um valor inteiro n ($1 \leq n \leq 100$) representando o número de nós, seguido por $n - 1$ ligações no formato $\{a, b\}$ ($1 \leq a, b \leq n$) separadas por um espaço entre uma e outra. Os casos de teste terminam com o fim da entrada.

Saída:

Para cada caso de teste deve ser impresso em uma linha única na saída a expressão “OK”, se não houver gato, ou “Miau” caso a árvore esteja mal formada (respeite maiúsculas e minúsculas nas expressões na saída).

Exemplo de entrada:

```
6 {1,4} {2,4} {3,4} {5,4} {5,6}
6 {1,4} {2,4} {3,6} {5,3} {5,6}
```

Saída do exemplo de entrada:

```
OK
Miau
```

Problema H
Halteres Equilibrados
Arquivo: halteres.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

A Academia Esportiva Sociedade dos Bons Competidores (SBC) é uma academia de ginástica e entre as atividades há o uso de halteres. Porém a academia é muito bagunçada e os “pesos” que são afixados nas duas pontas da barra do halteres estão todos espalhados e misturados, fica difícil achar 2 iguais para equilibrar. Principalmente quando queremos um conjunto maior, pois há muitos “leves” no meio. A opção é colocar vários e tentar equilibrar nos 2 lados. Por exemplo a distribuição seguinte é suficiente:

Esquerdo	10	15	3	2	30 kg
Direito	20	5	5		30 kg

De cada lado ficamos com 30kg, equilibrados um total de 60kg.

Por outro lado a distribuição a seguir não é boa:

Esquerdo	10	25	3	5	43 kg
Direito	20	5	5	10	40 kg

Neste caso o lado esquerdo é 3 quilogramas mais pesado que o direito. A você foi pedido que verifique se a distribuição Esquerdo - Direito colocado nos halteres está Ok.

Entrada:

Cada caso de entrada possui duas linhas, na primeira linha um inteiro E ($0 < E \leq 100$) e a seguir E valores inteiros e_i ($0 < e_i \leq 50$). Na segunda linha um inteiro D ($0 < D \leq 100$) e a seguir D valores inteiros d_i ($0 < d_i \leq 50$). Representando respectivamente a distribuição dos pesos no lado esquerdo e no direito. Todos os valores dos pesos são em quilogramas (kg).

Saida:

Para cada caso de entrada deve ser impresso em uma linha: “OK” se os pesos estiverem equilibrados. Ou “E > D: p kg” se o lado esquerdo for mais pesado que o direito por “p” quilogramas. Ou “D > E: p kg” caso o direito seja mais pesado.

Exemplo de entrada:

Saída do Exemplo de entrada:

4 10 15 3 2
3 20 5 5

OK

4 10 25 3 5
4 20 5 5 10

E > D: 3 kg

Problema I
Infinito Numérico
Arquivo: *infinito*.*[c|cpp|java|py]*
Limite de tempo: 1 s

O Problema:

Na matemática é muito comum tratarmos com o termo “*infinito*”, inclusive há um símbolo próprio para representá-lo: ∞ . O infinito em si é um número, o maior do sistema numérico adotado, que pode ser inteiro, ou real como exemplo. Há infinitos maiores que infinitos, mas não vamos nos prender a isso, e sim ao seu uso. Por exemplo a tendência de uma razão:

$$\lim_{x \rightarrow \infty} x \sin\left(\frac{1}{x}\right) = 1$$

Outro exemplo são integrais:

$$\int_{-\infty}^{\infty} e^{-ax^2} dx = \sqrt{\frac{\pi}{a}}$$

Mas o que nos interessa são as séries infinitas e suas somas. Como exemplo:

$$\sum_{n=1}^{\infty} \frac{1}{2^n} = 1$$

Muitas vezes precisamos calcular numericamente estas somas infinitas. Claramente não é possível somar *ad aeternum* então precisamos definir um infinito numérico. Por exemplo, na soma acima poderíamos considerar que a série atingiu a soma infinita se a diferença entre a soma atual e a soma com o próximo termo for menor que um erro previsto. Por exemplo se considerarmos o erro como 10^{-9} , atingimos o infinito no trigésimo termo, de fato: $1/2^{30} \approx 9.3 \times 10^{-10}$, ou seja incluir o trigésimo termo não muda a soma significativamente ($> 10^{-9}$) logo nosso infinito é $n = 30$

A você foi pedido para calcular a soma da série $\frac{a}{r^n}$ ($r > 1$) de $n = 1$ até $n = \infty$. Em especial apresentando o infinito numérico para a soma que atenda um determinado erro.

Entrada:

A entrada possui vários casos de teste, cada caso de teste é representado por 4 valores em uma linha: a um valor de ponto flutuante ($0 < a \leq 100$), r um valor de ponto flutuante ($1 < r \leq 100$), e um inteiro que representa a precisão do erro na forma 10^{-e} ($2 < e < 50$) e f um inteiro que representa a precisão em casas decimais com que queremos expressar a soma: ($0 < f < \min(e, 10)$). Os casos de teste terminam com o final da entrada.

Saida:

Para cada caso de teste deve ser impresso em uma linha única na saída um valor inteiro representando o infinito numérico e separado por um único espaço, um valor em ponto fixo com precisão definida por f representando a soma da Série considerando o infinito numérico encontrado. É garantido que o infinito numérico é menor que 10^5 .

Exemplo de entrada:

```
1 2 9 5
4.7 8 5 3
```

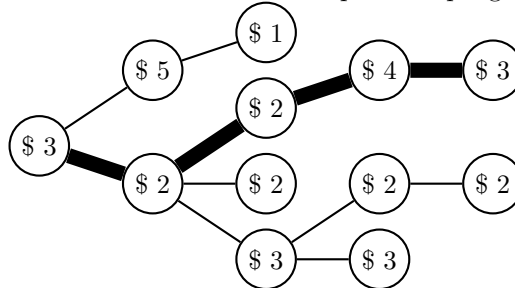
Saída do exemplo de entrada:

```
30 1.00000
7 0.671
```

Problema J
Jornada Milionária
Arquivo: *jornada.*[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

O canal de TV do Sistema Brasileiro de Competições (SBC) criou o programa Baú para Aquisição do Sucesso Efetivo (ProgBASE) onde há a competição Jornada Milionária. Nesta o competidor deve entrar por uma sala, desta há tuneis para outras salas (nunca podendo voltar) até uma sala sem saída. Em cada sala que passar ele encontrará um envelope indicando um prêmio em dinheiro. E na última sala ele irá acumular todos os prêmios que ganhou. Veja um exemplo:



No circuito acima, seguindo pelo caminho realçado em negrito, o competidor irá acumular o prêmio de \$ 14,00. Este seria o máximo possível. O diretor da ProgBASE pediu que você construísse um algoritmo onde, dado o circuito e prêmios disponíveis, você indique qual seria o maior valor possível que um competidor venha a receber.

Entrada:

A entrada possui vários casos de teste. Cada caso de teste começa com um valor inteiro S ($0 < S \leq 500$) o número de salas e um valor V em ponto flutuante ($0 < V < 50$) representando o prêmio em moedas (\$) associado à primeira sala. A primeira sala é sempre a sala de número 1. A seguir vem $S - 1$ linhas onde a i -ésima linha, representando a sala i estão dois números: P ($1 \leq P < i$) indicando qual a sala de origem para chegar nesta, e um valor V do prêmio da sala nos mesmos critérios do prêmio da primeira sala. O fim das entradas termina com $S = V = 0$ que não deve ser processado.

Saída:

Para cada caso de teste deve ser impresso em uma linha única o prêmio máximo acumulado possível no circuito, no formato “\$ v.vv” (um valor em ponto fixo de 2 casas precedido pelo símbolo \$ separado por um espaço).

Exemplo de entrada:

```
12 3.00
1 5.00
1 2.00
2 1.00
3 2.00
3 2.00
3 3.00
5 4.00
7 2.00
7 3.00
8 3.00
9 2.00
7 1.00
1 1.00
1 1.00
2 1.00
2 1.00
3 1.00
3 1.00
0 0
```

Saída do exemplo de entrada:

```
14.00
3.00
```